



CC1002 - Introducción a la Programación

CLASE 4 : BOOLEAN Y CONDICIONES

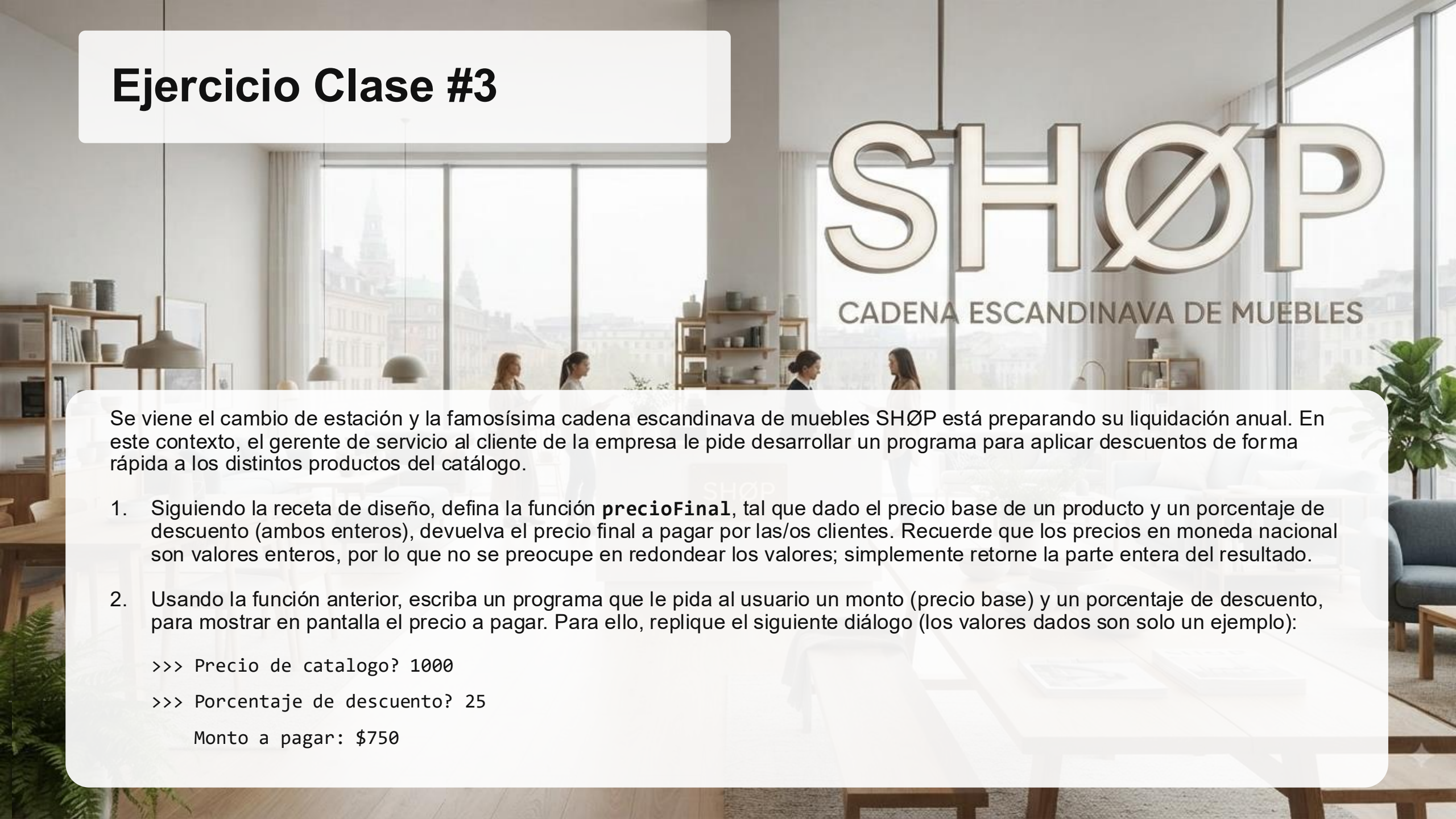
Departamento de Ciencias de la Computación (DCC) - FCFM
UNIVERSIDAD DE CHILE



Eduardo Godoy (egodoy@dcc.uchile.cl)

Nadia Decar (nadiadecar@ug.uchile.cl)

Ejercicio Clase #3

The background image shows a modern, bright interior of a furniture store. Large windows on the right side offer a view of a city with historic buildings. In the foreground, there are wooden shelves with various decorative items like vases and books. A large, three-dimensional logo for 'SHØP' is mounted on the wall, with the text 'CADENA ESCANDINAVA DE MUEBLES' underneath it. Several people are visible in the background, interacting in the store.

SHØP
CADENA ESCANDINAVA DE MUEBLES

Se viene el cambio de estación y la famosísima cadena escandinava de muebles SHØP está preparando su liquidación anual. En este contexto, el gerente de servicio al cliente de la empresa le pide desarrollar un programa para aplicar descuentos de forma rápida a los distintos productos del catálogo.

1. Siguiendo la receta de diseño, defina la función **precioFinal**, tal que dado el precio base de un producto y un porcentaje de descuento (ambos enteros), devuelva el precio final a pagar por las/os clientes. Recuerde que los precios en moneda nacional son valores enteros, por lo que no se preocupe en redondear los valores; simplemente retorne la parte entera del resultado.
2. Usando la función anterior, escriba un programa que le pida al usuario un monto (precio base) y un porcentaje de descuento, para mostrar en pantalla el precio a pagar. Para ello, replique el siguiente diálogo (los valores dados son solo un ejemplo):

```
>>> Precio de catalogo? 1000
```

```
>>> Porcentaje de descuento? 25
```

```
Monto a pagar: $750
```

Solución: precioFinal

```
# precioFinal: int int -> int
# Dado el precio base y un porcentaje de descuento,
# retorna el precio final a pagar (parte entera).
# Ejemplo: precioFinal(1000, 25) retorna 750
def precioFinal(precioBase, descuento):
    return int(precioBase * (100 - descuento) / 100)

assert precioFinal(1000, 25) == 750

# Programa principal
precio = int(input("Precio de catalogo? "))
dcto = int(input("Porcentaje de descuento? "))
print("    Monto a pagar: $" + str(precioFinal(precio, dcto)))
```

Ejercicio propuesto

Escriba la receta para 2 funciones:

a) Octaedro regular

$$V = (\sqrt{2} / 3) * a^3$$

a: longitud de la arista

b) Dodecaedro regular

$$V = (15 + 7\sqrt{5}) / 4 * a^3$$

a: longitud de la arista

- Indique, explicando, cómo se hace para que estas dos funciones puedan ser usadas por otro programa en Python.
- Escriba un ejemplo de uso de estas funciones en un programa.



Boolean

Tipo de dato lógico

- En computación se conoce como boolean o booleano
- True: Verdadero
- False: Falso

```
>>> True
True
>>> False
False
```



Comparación

Booleanos

```
>>> True == True
True
>>> False == False
True
>>> False == True
False
>>> False != True
True
```

Números

```
>>> 10 == 10
True
>>> 10 == 20
False
>>> 10 != 20
True
>>> 10 < 20
True
```



Comparación

Elementos para una comparación

== igual a

< menor que

!= distinto de

>= mayor o igual que

> mayor que

<= menor o igual que

Comparaciones compuestas

Operadores lógicos

and y lógico

or o lógico

not no lógico

```
>>> 1 < 2 and 3 <= 4
True
```

```
>>> 1 > 2 or 3 <= 4
True
```

```
>>> 1 > 2 or 3 > 4
False
```

```
>>> not (1 > 2 or 3 > 4)
True
```



Funciones booleanas

Una función puede indicar un estado booleano: True o False

PROBLEMA

Construir una función que indique si un número es par

RESTO DE LA DIVISIÓN

```
>>> 5 % 2  
1
```

```
>>> 4 % 2  
0
```



Funciones booleanas

Una función puede indicar un estado booleano: True o False

PROBLEMA

Construir una función que indique si un número es par

RESTO DE LA DIVISIÓN

```
>>> 5 % 2    →    1
>>> 4 % 2    →    0
```

Será par cuando el módulo sea cero

Funciones booleanas

```
# esPar: int -> boolean
# Dado un número, retorna True si el número es par.
# En caso contrario retorna False
# Ejemplo: esPar(3) retorna False
```

```
def esPar(numero):
    return (numero % 2) == 0
```

```
# Test
assert esPar(2) == True
assert esPar(5) == False
```



Funciones booleanas

```
# esPar: int -> boolean
# Dado un número, retorna True si el número es par.
# En caso contrario retorna False
# Ejemplo: esPar(3) retorna False
```

```
def esPar(numero):
    return (numero % 2) == 0
```

```
# Test
assert esPar(2)
assert not esPar(5)
```



Condicionantes

Tomar decisiones en un programa según una condición

`if`

`elif`

`else`



Condición: Problema

Dado dos números, ¿cuál es el más grande?

```
n1 = int(input('Ingresa un número? '))  
n2 = int(input('Ingresa un número? '))
```

? ? ?

```
print('El mayor es ', mayor)
```

**¡Aquí hay que tomar
una decisión!**



Condiciones: Solución 1

```
n1 = int(input('Ingresa un número? '))
n2 = int(input('Ingresa un número? '))

mayor = n1          # Se asume que n1 es mayor

if mayor < n2:      # pero hay que comprobarlo
    mayor = n2

print('El mayor es ', mayor)
```



Condiciones: Solución 2

```
n1 = int(input('Ingrese un número? '))
n2 = int(input('Ingrese un número? '))

if n1 < n2:
    mayor = n2
else:
    mayor = n1

print('El mayor es ', mayor)
```



Condiciones: Solución 3

```
n1 = int(input('Ingresa un número? '))
n2 = int(input('Ingresa un número? '))

if n1 > n2:
    print('El mayor es ', n1)
elif n2 > n1:
    print('El mayor es ', n2)
else:
    print('Ambos números son iguales', n1)
```



Condición: Problema

Dado tres números, ¿cuál es el más grande?

```
n1 = int(input('Ingrese un número? '))  
n2 = int(input('Ingrese un número? '))  
n3 = int(input('Ingrese un número? '))  
  
? ? ?  
  
print('El mayor es ', mayor)
```

Aquí hay que hacer algo...



Condiciones: Solución 1

```
n1 = int(input('Ingrese un número? '))
n2 = int(input('Ingrese un número? '))
n3 = int(input('Ingrese un número? '))

mayor = n1          # Se asume que n1 es mayor

if mayor < n2:      # hay que comprobarlo contra n2
    mayor = n2

? ? ?
    ? ? ?

print('El mayor es ', mayor)
```



Condiciones: Solución 1

```
n1 = int(input('Ingrese un número? '))
n2 = int(input('Ingrese un número? '))
n3 = int(input('Ingrese un número? '))

mayor = n1          # Se asume que n1 es mayor

if mayor < n2:      # comprobar contra n2
    mayor = n2

if mayor < n3:      # comprobar contra n3
    mayor = n3

print('El mayor es ', mayor)
```



Condiciones: Solución 2

```
n1 = int(input('Ingrese un número? '))
n2 = int(input('Ingrese un número? '))
n3 = int(input('Ingrese un número? '))

if n1 < n3 and n2 < n3:      # ¿Será n3 el mayor?
    mayor = n3
elif n1 < n2:                # ¿Será n2 el mayor?
    mayor = n2
else:
    mayor = n1                # n1 es el mayor

print('El mayor es ', mayor)
```



Condiciones: Solución 3

```
n1 = int(input('Ingresa un número? '))  
n2 = int(input('Ingresa un número? '))  
n3 = int(input('Ingresa un número? '))  
  
print('El mayor es ', max(n1, max(n2, n3)))
```

max · función ya definida en Python: entrega el mayor de dos valores



Condiciones: Solución 4

```
n1 = int(input('Ingresa un número? '))  
n2 = int(input('Ingresa un número? '))  
n3 = int(input('Ingresa un número? '))  
print('El mayor es ', max(n1, n2, n3))
```

max puede operar con n argumentos: `mayor = max(n1, n2, n3, n4, n5)`
También existe **min**, que entrega el menor.



Condición: Problema

Dado tres números, escríbalos de menor a mayor.

```
n1 = int(input('Ingrese un número? '))  
n2 = int(input('Ingrese un número? '))  
n3 = int(input('Ingrese un número? '))
```

? ? ?

```
print('El menor es ', menor)  
print('El medio es ', medio)  
print('El mayor es ', mayor)
```

**Aquí hay que establecer
el orden...**



Condiciones: Solución

```
n1 = int(input('Ingrese un número? '))
n2 = int(input('Ingrese un número? '))
n3 = int(input('Ingrese un número? '))

menor = min(n1, n2, n3)
mayor = max(n1, n2, n3)
medio = (n1 + n2 + n3) - menor - mayor

print('El menor es ', menor)
print('El medio es ', medio)
print('El mayor es ', mayor)
```



Condiciones y funciones

Construir la función **diasDelMes** que indica cuántos días tiene un mes.
Asuma, por el momento, que febrero tiene 28 días.

diasDelMes(1)
⇒ 31

diasDelMes(2)
⇒ 28

diasDelMes(4)
⇒ 30



Condiciones y funciones

```
# diasDelMes: int -> int
# dado el mes del año, retorna cuántos días tiene ese mes
# Ejemplo: diasDelMes(4) retorna 30
def diasDelMes(mes):
    if mes == 2:
        dias = 28
    elif mes == 1 or mes == 3 or mes == 5 or mes == 7 or \
         mes == 8 or mes == 10 or mes == 12:
        dias = 31
    elif mes == 4 or mes == 6 or mes == 9 or mes == 11:
        dias = 30
    else:
        dias = -1
    return dias
```



Condición: sintaxis

```
if expresión booleana :  
    instrucciones-1  
elif expresión booleana :  
    instrucciones-2  
elif expresión booleana :  
     $\vdots$   
else :  
    instrucciones-n
```



Solo if

¿Acaso «if» puede ir solo?

Sí, la instrucción **if** puede ir sola

```
n1 = int(input("Número 1? "))
n2 = int(input("Número 2? "))
mayor = n1
if mayor < n2:
    mayor = n2
print("El mayor es", mayor)
```



if + else

Si la condición del **if** es falsa, se ejecutan las instrucciones del **else**

```
n1 = int(input("Número 1? "))
n2 = int(input("Número 2? "))

if n1 < n2:
    mayor = n2
else:
    mayor = n1

print("El mayor es", mayor)
```

Importante

else no puede ir solo:
requiere que exista un **if**



| if + if

¿Puede un «if» contener otro «if»?

Sí: lo que contiene un **if** son instrucciones, y una de ellas puede ser otro **if**

```
edad = int(input("Dime tu edad? "))  
if (edad > 40):  
    print("Tienes más años que mi tío")  
    if (edad > 70):  
        print("¡Más que mi abuelo!")  
print("¡Gracias!")
```

{ Instrucciones del primer **if**



| if + if

¿Puede un «if» contener otro «if»?

Sí: lo que contiene un **if** son instrucciones, y una de ellas puede ser otro **if**

```
edad = int(input("Dime tu edad? "))  
if (edad > 40):  
    print("Tienes más años que mi tío")  
    if (edad > 70):  
        print("¡Más que mi abuelo!")  
print("¡Gracias!")
```

{ Instrucciones del segundo **if**



| if + if + else

¿Puede un «if» contener otro «if» con «else»?

Sí: un **if** contiene instrucciones, y una de ellas puede ser otro **if + else**

```
edad = int(input("Dime tu edad? "))
if (edad > 40):
    print("Tienes más años que mi tío")
    if (edad > 70):
        print("¡Más que mi abuelo!")
    else:
        print("Menos que mi abuelo")
print("¡Gracias!")
```

} Instrucciones del primer **if**



if + else + if versus if + elif

if + else + if

```
edad = int(input("Tu edad? "))  
if edad < 40:  
    print("Eres menor que mi tío")  
else:  
    if edad > 70:  
        print("Eres mayor que mi abuelo")
```

if + elif

```
edad = int(input("Tu edad? "))  
if edad < 40:  
    print("Eres menor que mi tío")  
elif edad > 70:  
    print("Eres mayor que mi abuelo")
```

¡Ambos códigos son similares!



Verdadero o Falso

```
if condición_1 :  
    instrucción_1a  
    instrucción_1b  
    if condición_2 :  
        instrucción_2a  
        instrucción_2b  
    instrucción_1c  
else:  
    if condición_3 :  
        instrucción_3a  
        instrucción_3b  
    instrucción_d  
instrucción_e
```

¿Qué instrucciones se ejecutan en cada caso?

1a	1b	2a	2b	1c	3a	3b	d	e



Verdadero o Falso

```
if condición_1 :  
    instrucción_1a  
    instrucción_1b  
    if condición_2 :  
        instrucción_2a  
        instrucción_2b  
    instrucción_1c  
else:  
    if condición_3 :  
        instrucción_3a  
        instrucción_3b  
    instrucción_d  
instrucción_e
```

Caso 1:

condición_1 $\Rightarrow$ True
condición_2 $\Rightarrow$ True
condición_3 $\Rightarrow$ True

1a	1b	2a	2b	1c	3a	3b	d	e



Verdadero o Falso

```
if condición_1 :  
    instrucción_1a  
    instrucción_1b  
    if condición_2 :  
        instrucción_2a  
        instrucción_2b  
    instrucción_1c  
else:  
    if condición_3 :  
        instrucción_3a  
        instrucción_3b  
    instrucción_d  
instrucción_e
```

Caso 1:

condición_1 $\Rightarrow$ True
condición_2 $\Rightarrow$ True
condición_3 $\Rightarrow$ True

1a	1b	2a	2b	1c	3a	3b	d	e
✓	✓	✓	✓	✓				✓



Verdadero o Falso

```
if condición_1 :  
    instrucción_1a  
    instrucción_1b  
    if condición_2 :  
        instrucción_2a  
        instrucción_2b  
    instrucción_1c  
else:  
    if condición_3 :  
        instrucción_3a  
        instrucción_3b  
    instrucción_d  
instrucción_e
```

Caso 2:

condición_1 $\Rightarrow$ True
condición_2 $\Rightarrow$ False
condición_3 $\Rightarrow$ True

1a	1b	2a	2b	1c	3a	3b	d	e
✓	✓	✓	✓	✓				✓

1a	1b	2a	2b	1c	3a	3b	d	e



Verdadero o Falso

```
if condición_1 :  
    instrucción_1a  
    instrucción_1b  
    if condición_2 :  
        instrucción_2a  
        instrucción_2b  
    instrucción_1c  
else:  
    if condición_3 :  
        instrucción_3a  
        instrucción_3b  
    instrucción_d  
instrucción_e
```

Caso 1

condición_1 $\Leftarrow$ True
condición_2 $\Leftarrow$ True
condición_3 $\Leftarrow$ True

Caso 2

condición_1 $\Leftarrow$ True
condición_2 $\Leftarrow$ False
condición_3 $\Leftarrow$ True

1a	1b	2a	2b	1c	3a	3b	d	e
✓	✓	✓	✓	✓				✓

1a	1b	2a	2b	1c	3a	3b	d	e
✓	✓			✓				✓



Verdadero o Falso

```
if condición_1 :  
    instrucción_1a  
    instrucción_1b  
    if condición_2 :  
        instrucción_2a  
        instrucción_2b  
    instrucción_1c  
else:  
    if condición_3 :  
        instrucción_3a  
        instrucción_3b  
    instrucción_d  
instrucción_e
```

Caso 1 $c1 \Leftarrow \text{True} \cdot c2 \Leftarrow \text{True} \cdot c3 \Leftarrow \text{True}$

1a	1b	2a	2b	1c	3a	3b	d	e
✓	✓	✓	✓	✓				✓

Caso 2 $c1 \Leftarrow \text{True} \cdot c2 \Leftarrow \text{False} \cdot c3 \Leftarrow \text{True}$

1a	1b	2a	2b	1c	3a	3b	d	e
✓	✓			✓				✓

Caso 3 $c1 \Leftarrow \text{False} \cdot c2 \Leftarrow \text{True} \cdot c3 \Leftarrow \text{True}$

1a	1b	2a	2b	1c	3a	3b	d	e



Verdadero o Falso

```
if condición_1 :  
    instrucción_1a  
    instrucción_1b  
    if condición_2 :  
        instrucción_2a  
        instrucción_2b  
    instrucción_1c  
else:  
    if condición_3 :  
        instrucción_3a  
        instrucción_3b  
    instrucción_d  
instrucción_e
```

Caso 1 $c1 \Leftarrow \text{True} \cdot c2 \Leftarrow \text{True} \cdot c3 \Leftarrow \text{True}$

1a	1b	2a	2b	1c	3a	3b	d	e
✓	✓	✓	✓	✓				✓

Caso 2 $c1 \Leftarrow \text{True} \cdot c2 \Leftarrow \text{False} \cdot c3 \Leftarrow \text{True}$

1a	1b	2a	2b	1c	3a	3b	d	e
✓	✓			✓				✓

Caso 3 $c1 \Leftarrow \text{False} \cdot c2 \Leftarrow \text{True} \cdot c3 \Leftarrow \text{True}$

1a	1b	2a	2b	1c	3a	3b	d	e
					✓	✓	✓	✓



Ejercicio propuesto

15 minutos

1. Escribir la función `cachipunBBTheory` que reciba un número entero y entregue "piedra", "papel", "tijeras", "Spock", "lagarto" o "error" si el número es 1, 2, 3, 4, 5 u otro. Debe incluir la receta de diseño con 2 casos de prueba.
2. Usar esa función en un programa interactivo que permita jugar entre una persona y el computador, como en el diálogo de ejemplo.

Bienvenido a Cachipún Big Bang Theory.

Las posibles jugadas son `piedra(1)`,
`papel(2)`, `tijeras(3)`, `Spock(4)`,
`lagarto(5)`.

¿Cuál es tu opción? 2

Ud jugó `papel`

Computador jugó `tijeras`

Gana computador

Notas

- Las otras respuestas posibles son "Gana Ud" o "Empate", según lo que jueguen la persona y el computador.
- El computador juega con `random.randint(1, 5)`; hay que importar el módulo `random`.
- Puede personalizar los mensajes.

